

Summer term 2026

Visual Data Analysis

Assignment Sheet 10

Solution has to be submitted via eCampus by June 26, 2026, 10:00 a.m.

If you have questions concerning the exercises, please use the forum on eCampus.

- Please work on this exercise in **small groups** of 3 students. Submit each solution only once, but clearly indicate who contributed to it by forming a team in eCampus. Remember that all team members have to be able to explain all answers.
- Please submit your answers in PDF format, and your scripts as *.py/*.ipynb files. If you are using [Jupyter notebook](#), please also export your scripts and results as PDF.

Exercise 1 (Isosurface Post-Processing in ParaView, 9 Points)

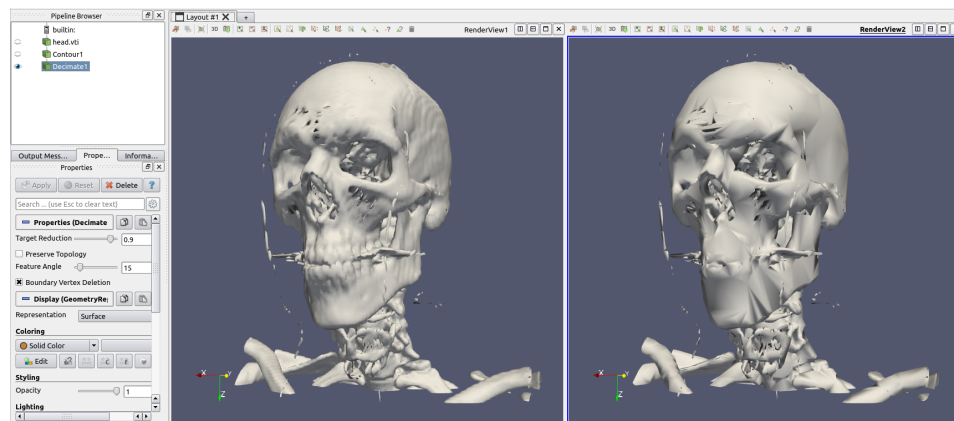


Figure 1: Left: Original Mesh, Right: Decimated Mesh.

In this task, you will explore different ways of post-processing an isosurface mesh using the software ParaView, which you got to know on the previous sheet.

- a) Read the file `head.vti` from the previous sheet and use the **Contour** filter to extract an isosurface similar to the one on the left of Fig. 1. (1P)
- b) By default, the surface is shaded using normals that the Marching Cubes algorithm computes from gradients of the underlying volume dataset. Explore the effect of replacing those with normals estimated from the triangular mesh itself, by adding a **SurfaceNormals** filter. For easier comparison, set up ParaView so that you can view both versions side by side. To achieve this, split the viewer window and create a second **RenderView** whose camera should be linked to the existing view (right click in one view, “Link camera...”, click on the other view). Then, use the eye icons in the Pipeline Browser to decide which version of the mesh should be visible in each **RenderView**. Describe your observation. (2P)

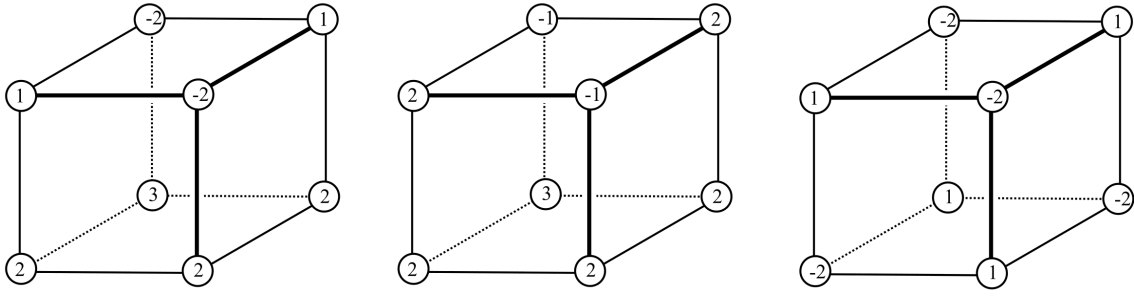


Figure 2: Your task is to apply Marching Cubes to these cells.

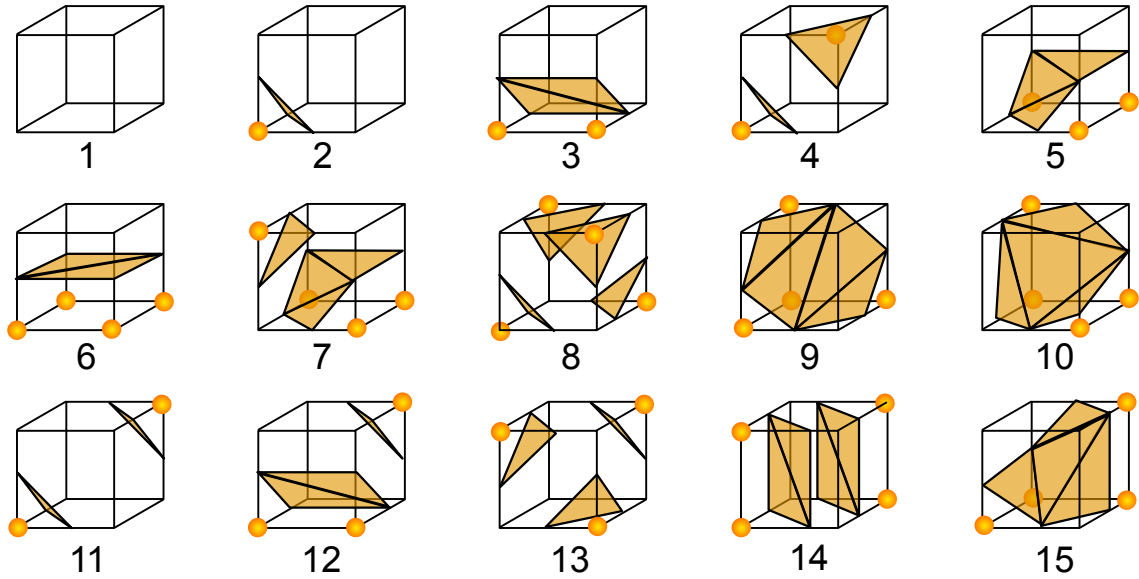


Figure 3: Marching Cubes cases. Please remember that, depending on the outcome of the decoder, some cases might result in different triangles than shown in this table.

- c) The **SurfaceNormals** filter supports detection of feature edges, as we discussed them in the context of mesh decimation. By default, the feature angle is set to 30° . Change this setting to 180° and describe your observations. (1P)
- d) Use the **Smooth** filter to apply 100 iterations of Laplacian smoothing to the outcome of the **SurfaceNormals** filter. Observe how this changes the rendering. Add another **SurfaceNormals** filter after the smoothing. Describe and explain your observations. (2P)
- e) Add a **Decimate** filter to reduce the number of triangles in the original mesh. Based on visual inspection, does decimation have any desirable effects other than reducing the number of triangles? What would you consider as the largest reduction factor that maintains all relevant features of the shape? Submit a screenshot in which you highlight a visible artifact or a lost feature at a larger reduction rate. (3P)

Exercise 2 (Marching Cubes, 9 Points)

Given the cells in Figure 2, apply the Marching Cubes algorithm for the isovalue $c = 0$ by drawing the resulting triangles of the isosurface. Indicate which of the 15 base cases shown in Figure 3 has to be

used to connect the edge intersections in each case. Mark the faces of the cells that result in ambiguous configurations and apply the asymptotic decider where needed.

Exercise 3 (Direct Volume Rendering in ParaView, 10 Points)

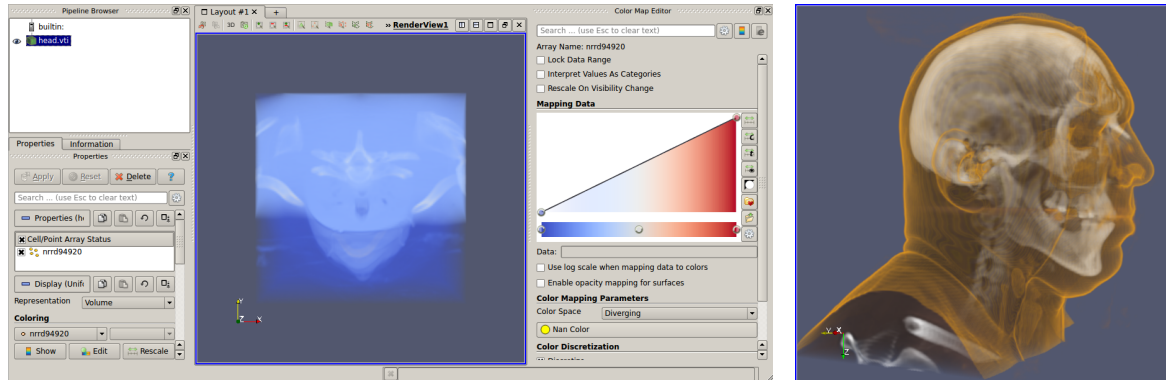


Figure 4: Initial direct volume rendering in ParaView (left) and result after adapting the transfer function and rotation (right)

In this task, you are going to learn how to perform direct volume rendering in ParaView.

- Open ParaView and load the `head.vti` dataset from the previous sheet. In the “Properties” tab, hit “Apply” to actually make it available. Now, change the representation to “Volume” and confirm your choice. At this point, you should see a result as in Figure 4 left. Now, modify the transfer function and rotate the volume so that you obtain a result similar to Figure 4 right. Please submit a screenshot of your result, including the transfer function editor. (3P)
- In Figure 5, a transfer function has been chosen that assigns full opacity, but only in a very narrow value range (close to a delta peak). What might be the intended effect of such a transfer function? Why does it not lead to cleaner image? Which method do you remember from the lecture that could improve the rendering in such a case? (3P)
- Figure 6 left shows the result of using a step function as a transfer function, which visualizes the silhouette of a surface. Provide a screenshot that additionally reveals the surface shape, as in the middle image. What was missing in the first case, and had to be added for the second one? (2P)
- While rotating the volume, you will notice artifacts such as the ones highlighted by the red arrow in Figure 6 right. Explain why they vanish automatically once you release the mouse. (2P)

Exercise 4 (Rendering Speed of Direct Volume Rendering in VTK, 15 Points)

In this task, you will use one of the direct volume rendering implementations that are available in VTK via its Python bindings. This permits a fine-grained control over its settings, and can be used to investigate their effect on rendering speed.

We provide a framework `SimpleRayCast.py` that sets up the renderer, counts the number of frames that are rendered during one second, and reports the resulting rendering speed in frames per second on the terminal. Use it to render the volume `bonsai.vti` that we provide via eCampus, and perform the following tasks:

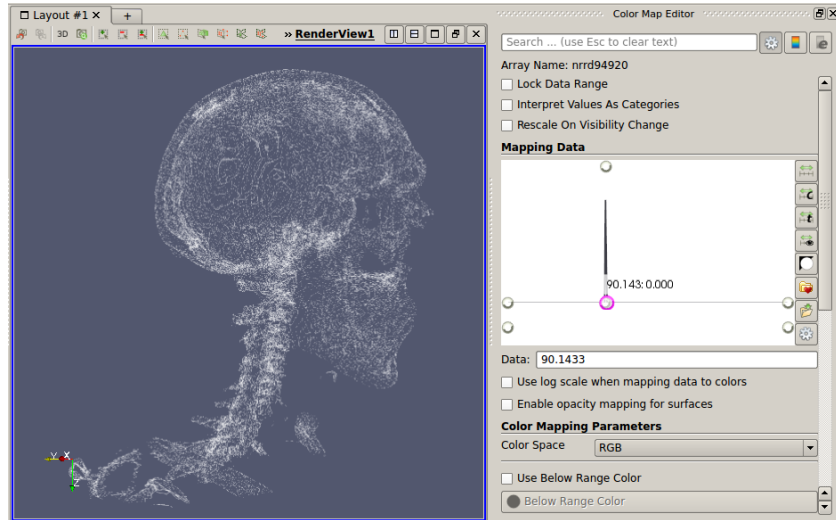


Figure 5: Result from a transfer function that is close to a delta peak.

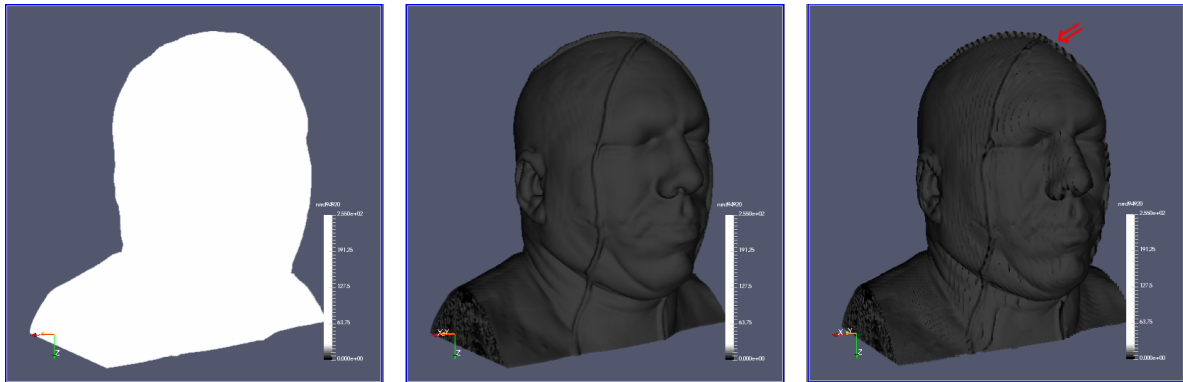


Figure 6: Silhouette rendering (left), same surface with details (center), and artifacts during interaction (right).

- Rotate the volume for multiple seconds and note the resulting rendering speed. Repeat your experiment after zooming out, and once more after zooming in. Explain your observations. (2P)
- Our framework provides an infrastructure to toggle between transfer functions by pressing `t`, but only implements a single transfer function by default. Add another transfer function that only assigns non-zero opacity in a narrower value range, to achieve a cleaner rendering, similar to Figure 7 (right). How does this affect rendering speed? Explain your observations based on the documentation for the `vtkFixedPointVolumeRayCastMapper`. (5P)
- Extend the framework so that pressing `i` toggles between trilinear and nearest neighbor interpolation. Describe and explain how this affects rendering speed. What else do you notice? (5P)
- Our framework disables the `AutoAdjustSampleDistances` property of the volume renderer. Turn it back on and repeat the previous experiments. How did this affect the rendering speed? What else do you notice? Again, explain your observations based on the documentation. (3P)



Figure 7: Rendering with the default transfer function of our framework (left) and potential result after transfer function refinement (right).

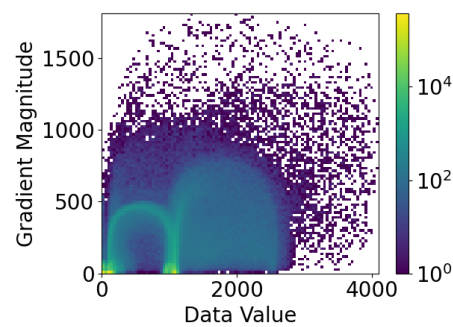


Figure 8: 2D Histogram of an example volume.

Exercise 5 (Transfer Functions, 7 Points)

- Given the 2D histogram shown in Figure 8, how many different materials do you expect to find in the corresponding volume? (1P)
- Do you expect that all pairwise boundaries between materials have similar area, or do you mostly see boundaries between specific materials? (2P)
- Do you expect that a 3D transfer function could be helpful to visualize this volume, or could a 2D transfer function be sufficient in this case? (2P)
- Transfer functions can be understood as the composition of two separate functions. When adding a shading model to direct volume rendering, which of these two stages is affected and why? (2P)

Good Luck!